

PROJECT AND TEAM INFORMATION

Project Title

Disaster Resource Allocation System

Student / Team Information

Team Name: Team DSCPP-III-2026-T100	Sem 3rd
Team member 1 (Team Lead)	Bisht, Shubham – 2510012245 (Sec F) bishtshubham265@gmail.com 
Team member 2	Bhandari, Samiksha – 2510010580 (Sec DS1) bhandarisamiksha22@gmail.com 
Team member 3	Semwal, Sarthak– 2510010180 (Sec ML4) sarthaksemwal92@gmail.com 
Team member 4	Rawat, Nikhil Singh – 2510012376 (Sec DS2) nikhil.singh0072@gmail.com 

PROPOSAL DESCRIPTION

Motivation

- *The big problem is that this manual way is slow and breaks down quickly, leaving one camp with extra supplies while a nearby camp runs out.*
- *In a time of crisis, this mismatch wastes people precious time they don't have.*
- *This project really aims to automate decisions for who needs help most urgently and who is closest, saving critical time.*

State of the Art / Current solution

- *Right now, this is mostly handled in the old fashioned way by phone calls, radio between camps, NGOs but they don't have the right technology.*
- *There are logistics and inventory tools out there but they are built for warehouses and supply chains in normal conditions, not for a situation changing by the hour that is why we need this type of service.*
- *Current softwares ignores live urgency, sudden road closure, availability of volunteer in real time.*
- *Decision making depends mostly on a single person who is incharge which is a lot of burden for one person to make decisions at these situations.*

Project Goals and Milestones

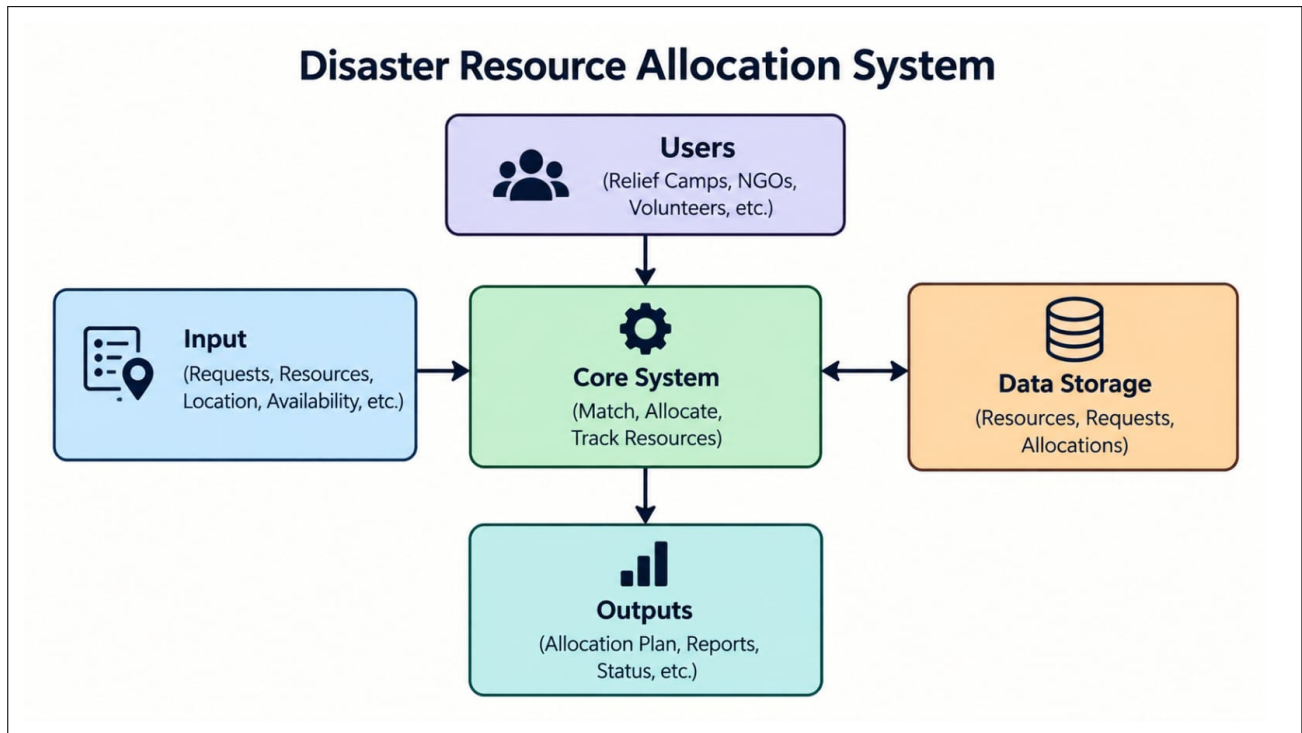
Our goal, in short: build something that takes in relief requests and available resources and figures out on its own who gets what first based on urgency and distance instead of someone having to work that out by hand under pressure.

Timeline	What we will build
Phase I (28/08/2026 20/09/2026)	We are starting with the groundwork writing the core classes – (Request, Resource, Volunteer, Hub), setting up the road network as a graph, and getting our priority queue working so requests are already being ranked by urgency from the very first version.
Phase II (21/09/2026 15/10/2026)	This is where it actually starts doing something . We will wire up Dijkstra's algorithm to find shortest routes, build the logic that matches requests to the nearest hub with stock, and hook up a database so nothing gets lost between runs.
Phase III (16/10/2026 31/10/2026)	Once the backend is working,we want to actually see it working,so we are building a CLI where you can watch dispatch happen live, see the queue re ranking itself, and even simulate a road closing to see how the system reacts.
Phase IV (01/11/2026 20/11/2026)	At Last we are throwing tough scenarios at it like a bridge going down mid crisis or a sudden spike in requests,cleaning up memory leaks,and taking everything together into our report and demo.

Project Approach

- *Deciding who's most urgent – Max-Heap Priority Queue:* every request that comes in gets pushed into a max-heap ranked by urgency, so whoever needs help most is always sitting right at the top, ready to be pulled off first. No scanning through the whole list every time something new arrives.
- *Mapping the roads :* we are treating relief hubs and affected zones as points on a map, connected by roads , stored as an adjacency list without having to rebuild the whole thing.
- *Finding the fastest route using Dijkstra's Algorithm:* once a request gets pulled off the priority queue, we run Dijkstra on that graph to actually work out the shortest path from a hub to the zone that needs help .
- *Deciding which hub sends the resources :* rather than checking every single hub against every single request, we go with a greedy approach. Assign the closest hub that actually has enough stock, which keeps things fast even when a lot of requests are coming at once.
- *Checking what is in stock using Hashing:* each hub's inventory sits in a hash table, so when the matching logic needs to know that does this hub have enough medical kits right now, it gets an answer almost instantly instead of searching through records.
- *Keeping track of pending requests using Linked List:* requests that are waiting their turn get chained together in a linked list, so we can add or remove one from the middle without changing anything else.
- *Modeling everything as objects – OOP (Classes, Inheritance, Polymorphism):* Request, Resource, Volunteer, and Hub are all classes; Resource branches into Food, Medical, and Shelter, and each one overrides a shared `allocate()` method, so the rest of the system doesn't need to care which type it's handling.
- *Containers we are leaning on:* vector for our lists of hubs and requests, unordered_map for fast inventory lookups, and priority_queue for the triage heap.
- *Where the data lives:* SQLite (or a simple file-based store, depending on how far we get) so inventory, requests, and past allocations don't just disappear when the program closes.
- *What we're building it with:* C++ in VS Code, with Git/GitHub so all four of us are not stepping on each other's code.
- *How we're working:* above design first, then build, then make it usable, then stress-test it before we call it done.

System Architecture (High Level Diagram)



Project Outcome / Deliverables

- By the end of this, we want a working prototype of the Disaster Resource Allocation System, not just a design on paper.
- It takes in and ranks requests: distress calls or aid requests come in and get sorted automatically by how urgent they are.
- It figures out who's got the goods : it looks for the closest relief center or warehouse that actually has what's needed, in stock, right now.
- It tells you exactly what to do next : for every dispatch cycle, it lays out what being sent, where its going, who is getting it, and the route to take.
- Alongside the prototype itself, here's what we will be handling in:

Source code - the actual application doing the prioritizing and matching.

Database schema - how we have organized requests, inventory, warehouses and routes.

System architecture write-up - how everything fits together and talks to each other.

Test scenarios with results - proof it actually holds up across different situations and not just the happy path.

Final presentation – a walkthrough of the problem, how we solved it, a live demo and what we found.

Assumptions

- *We are assuming relief center locations and their stock levels are known ahead of time and can actually be entered into the system when needed.*
- *We are assuming requests come in with at least a rough location and some sense of how urgent they are . we are not trying to solve for zero information and unknown problems.*
- *We are treating the road network between zones as a graph with distances that stay roughly the same, or change slowly enough that we don't need live traffic data cause it will make it more complex.*

References

<https://github.com/Rayan-Ali1083/Disaster-Relief-Management-System>
<https://github.com/RutujaBelokar08/Disaster-Relief-Resources-Allocation-System>